

Page table implementation

Module 5

Self study material

DSP (1DT003) & OSPP (1DT096)

Spring 2026

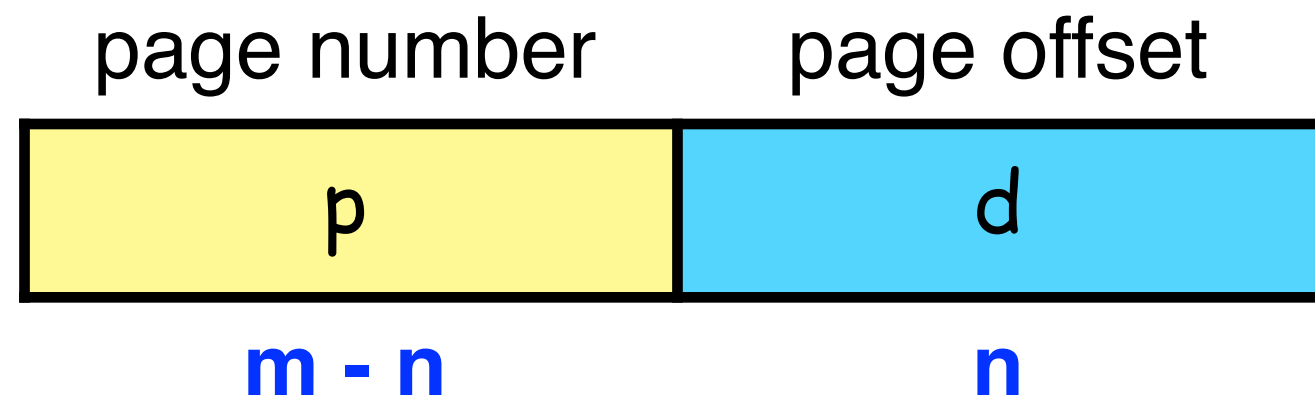
Paged memory management

- ▶ **Logical address space** of a process can be **noncontiguous**; process is **allocated physical memory whenever** the latter is **available**.
- ▶ Divide **physical memory** into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8,192 bytes).
- ▶ Divide **logical memory** into blocks of same size called **pages**.
- ▶ Keep track of all **free frames**.
 - To run a program of size n pages, need to find n free frames and load program.
- ▶ Set up a **page table** to translate logical to physical addresses.
- ▶ **Internal fragmentation** possible.
- ▶ No **external fragmentation**.

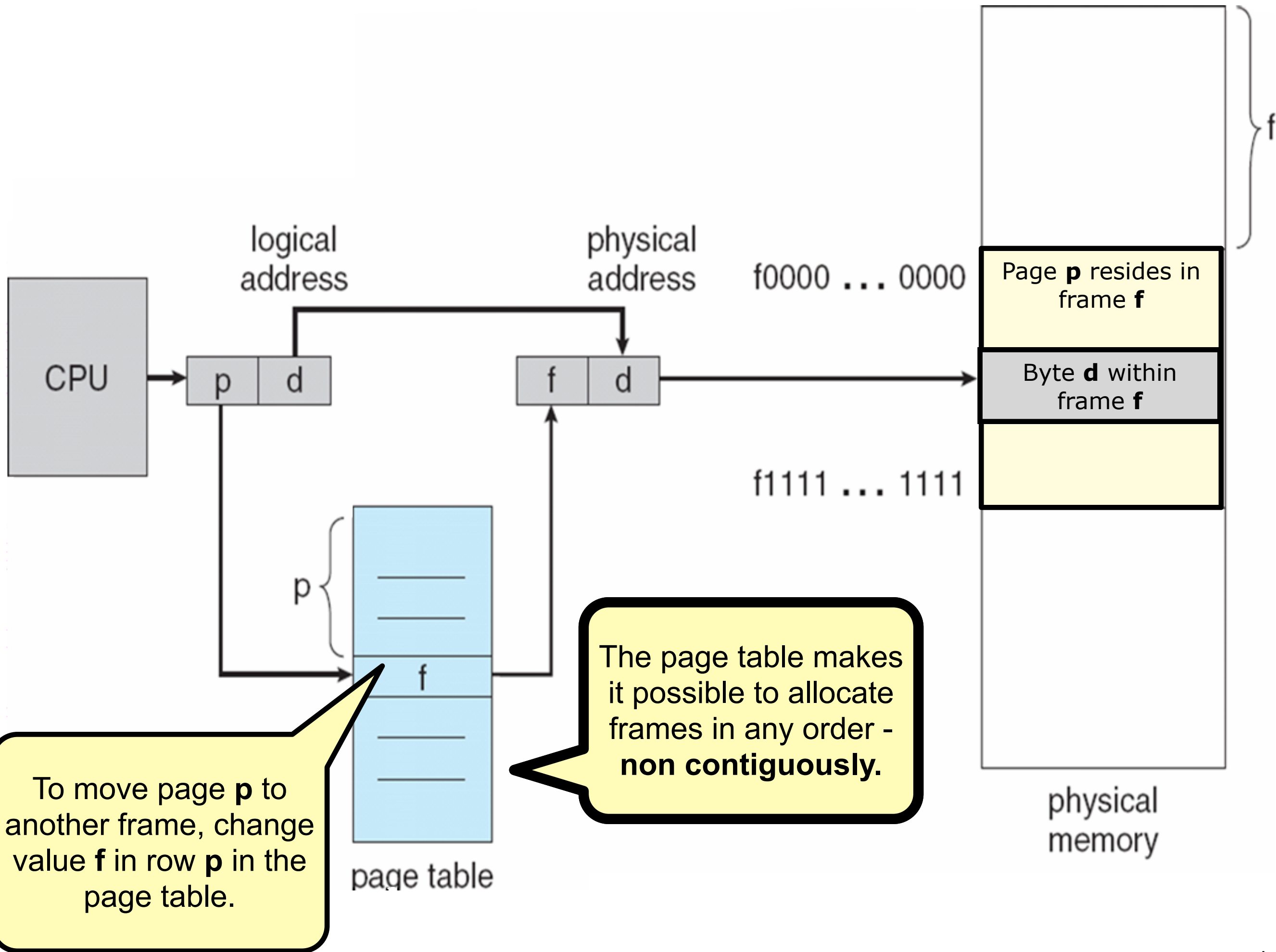
Address translation scheme

Logical address generated by CPU is divided into:

- ▶ **Page number** (p) – used as an index into a page table which contains base address of each frame in physical memory
- ▶ **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit.



For given logical address space 2^m and page size 2^n



- ▶ Page table is kept **in main memory**.
 - ▶ Page-table **base register** (PTBR) points to the page table.
 - ▶ Page-table **length register** (PRLR) indicates size of the page table.
- ▶ In this scheme every data/instruction access requires two memory accesses.
 - One for the page table.
 - One for the data/instruction.
 - ▶ Can we do better than this?

Binary prefixes

A binary prefix is a unit prefix for multiples of units in data processing, data transmission, and digital information, notably the bit and the byte, to indicate multiplication by a power of 2.

Prefixes for decimal and binary multiples

Decimal				Binary				
Value		SI		Value	IEC	JEDEC		
1000	10^3	k	kilo	1024	2^{10}	Ki kibi	K	kilo
1000^2	10^6	M	mega	1024^2	2^{20}	Mi mebi	M	mega
1000^3	10^9	G	giga	1024^3	2^{30}	Gi gibi	G	giga
1000^4	10^{12}	T	tera	1024^4	2^{40}	Ti tebi	T	tera
1000^5	10^{15}	P	peta	1024^5	2^{50}	Pi pebi	—	
1000^6	10^{18}	E	exa	1024^6	2^{60}	Ei exbi	—	
1000^7	10^{21}	Z	zetta	1024^7	2^{70}	Zi zebi	—	
1000^8	10^{24}	Y	yotta	1024^8	2^{80}	Yi yobi	—	
1000^9	10^{27}	R	ronna			—		
1000^{10}	10^{30}	Q	quetta			—		

Large address space

Most modern computer systems support a large logical address space, usually with 2^{32} - 2^{64} bytes of addressable memory.

Example

A system uses 32 bit logical memory addresses

Suppose the page size is 4 KB

$$4 \text{ KB} = 2^2 \times 2^{10} = 2^{12} \text{ Bytes}$$

Number of pages?

Symbol	Prefix	Factor	Factor
P	peta	2^{50}	1125899906842624
T	tera	2^{40}	1099511627776
G	giga	2^{30}	1073741824
M	Mega	2^{20}	1048576
k	kilo	2^{10}	1024

$$N = 2^{32} / 2^{12} = 2^{20} = 1048576 \approx 1 \text{ Million pages}$$

Assume each entry in the page table takes 4 bytes

$$\text{Size of page table} = 4 \times 2^{20} = 4 \text{ MB.}$$

Observation?

This is quite a lot of memory ... needed by every process!



If we try to allocate the page table contiguously in memory we will encounter the same problem with fragmentation that we try to solve using page tables in the first place.

Data structures for page tables

To overcome the problem with allocating a large contiguous piece of memory for the page table various alternative data structures can be used.

Structure of the page table

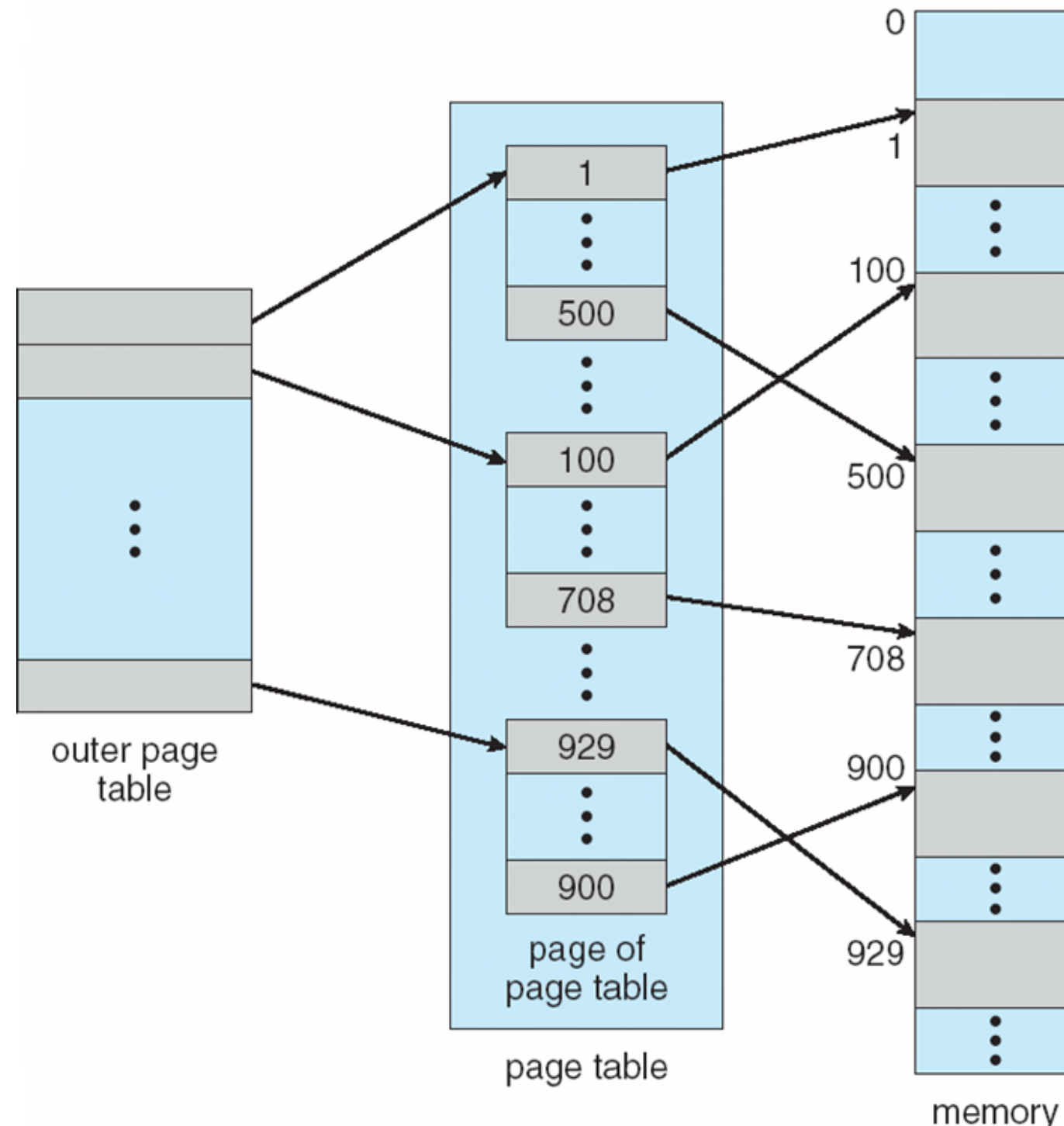
The page table can be structured in various ways.

- ▶ Hierarchical paging
- ▶ Hashed page tables
- ▶ Inverted page tables

Hierarchical page tables

Hierarchical page tables

Break up the logical address space into multiple page tables. A simple technique is a two-level page table.



Two-level paging example

A logical address on a 32-bit machine with 1KB page size is divided into:

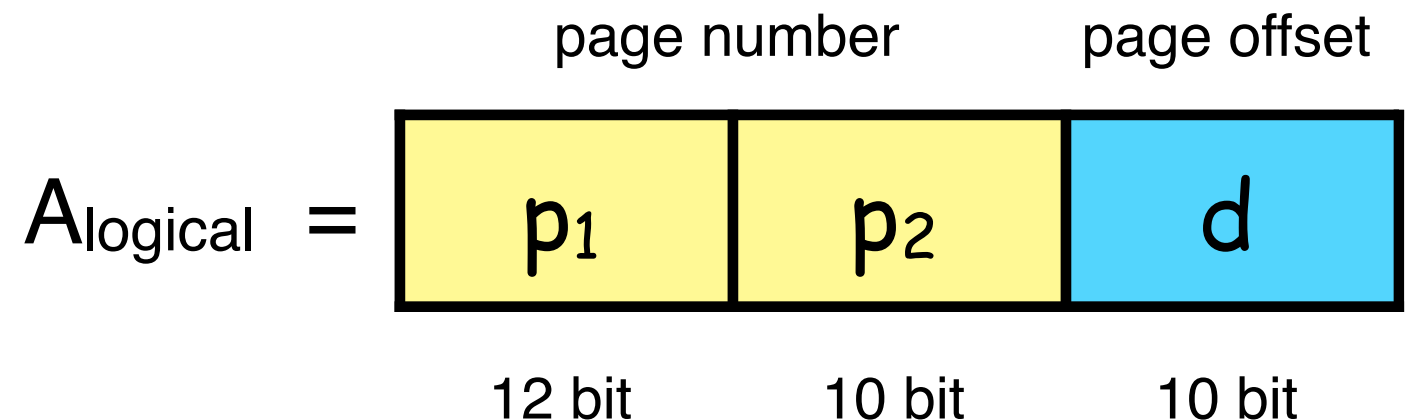
- ▶ a page number consisting of 22 bits
- ▶ a page offset consisting of 10 bits

Page size = 1KB = 2^{10} Bytes

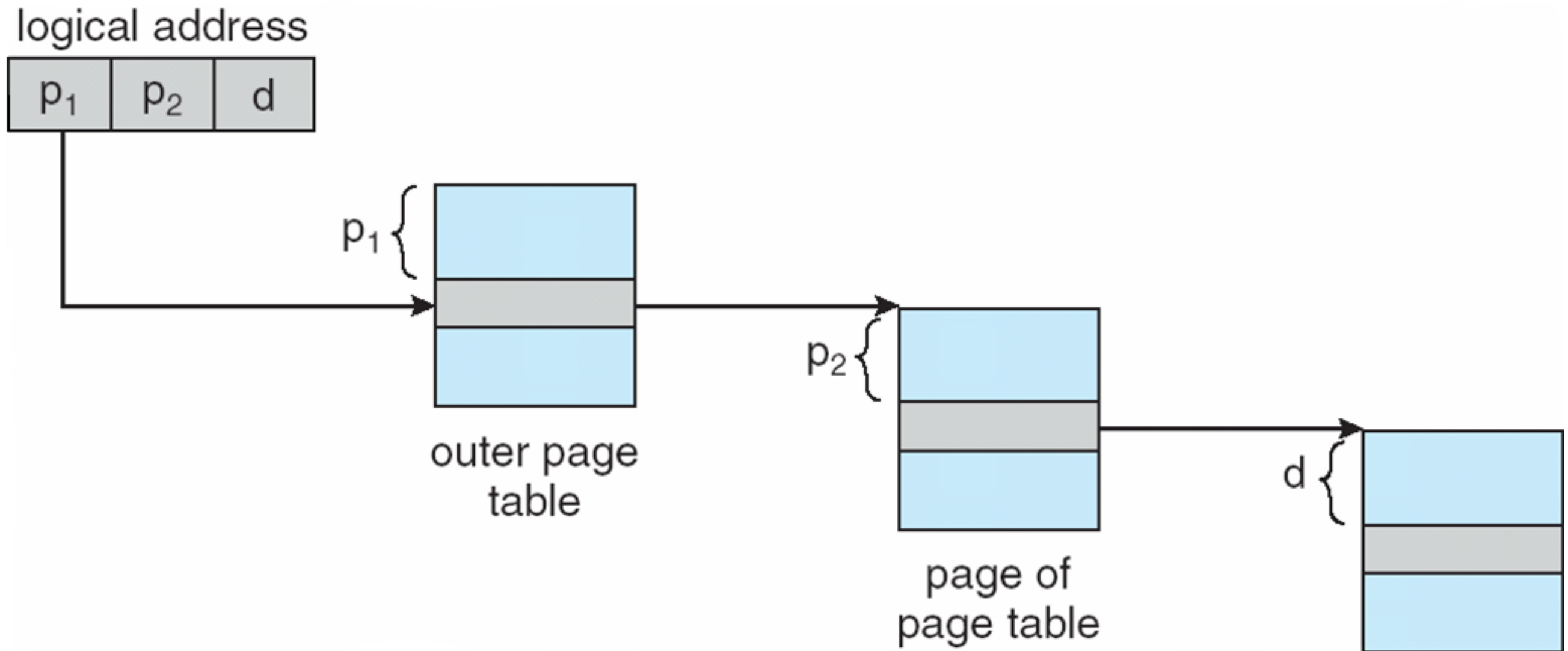
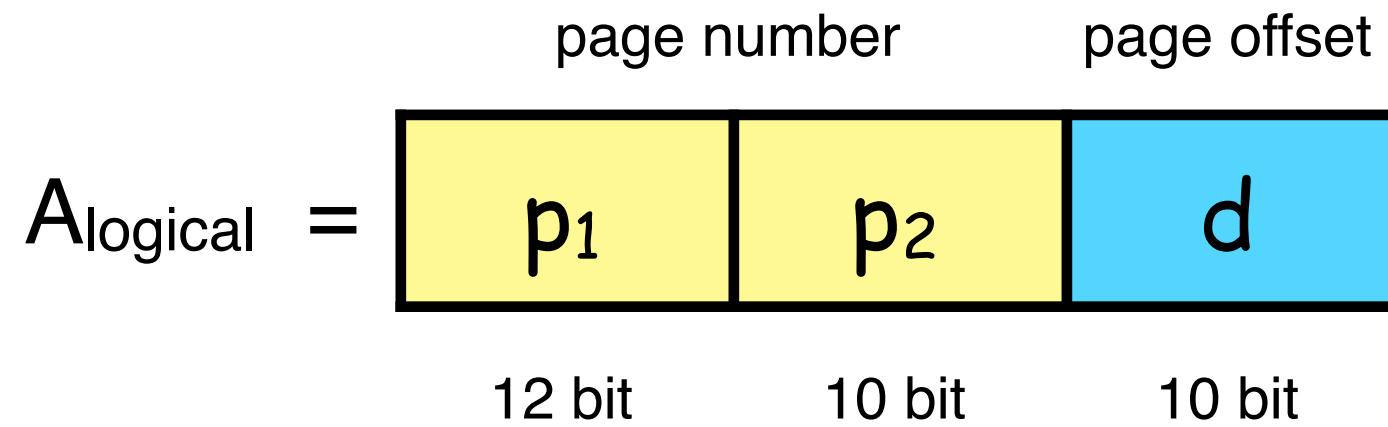
$N_{\text{pages}} = 2^{32}/2^{10} = 2^{22}$

Since the page table is paged, the 22 bit page number is further divided into:

- ▶ a 12-bit page number
- ▶ a 10-bit page offset

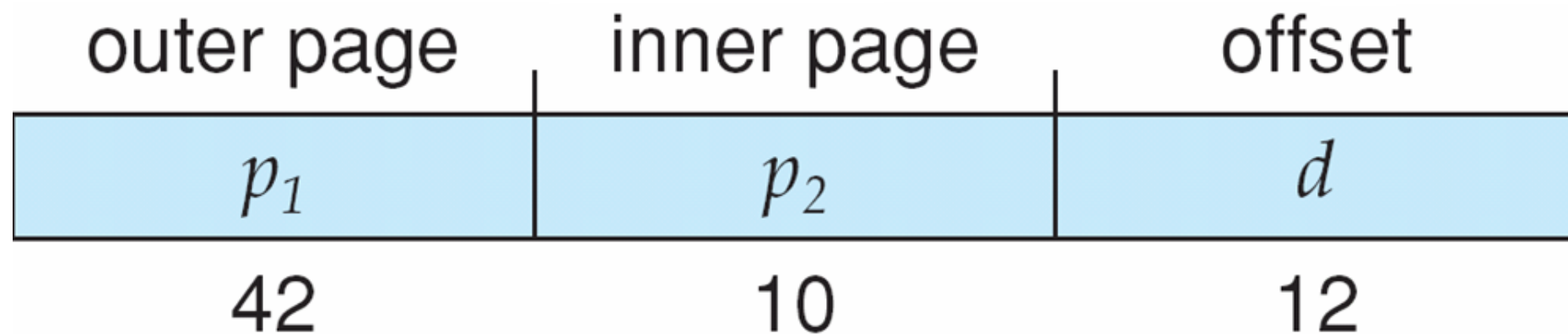


Where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the outer page table

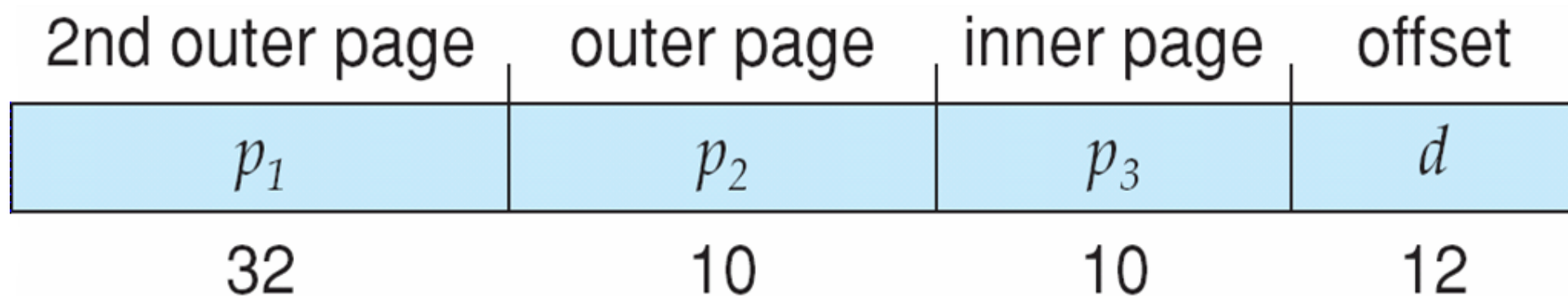


64 bit address space examples

Two Level Paging



Three Level Paging



Hash function

A hash function is any function that can be used to map data of arbitrary size to data of fixed size.

The values returned by a hash function are called hash values.

Typically, the domain of a hash function is larger than its range, and so it will map several different keys to the same index which could result in collisions.

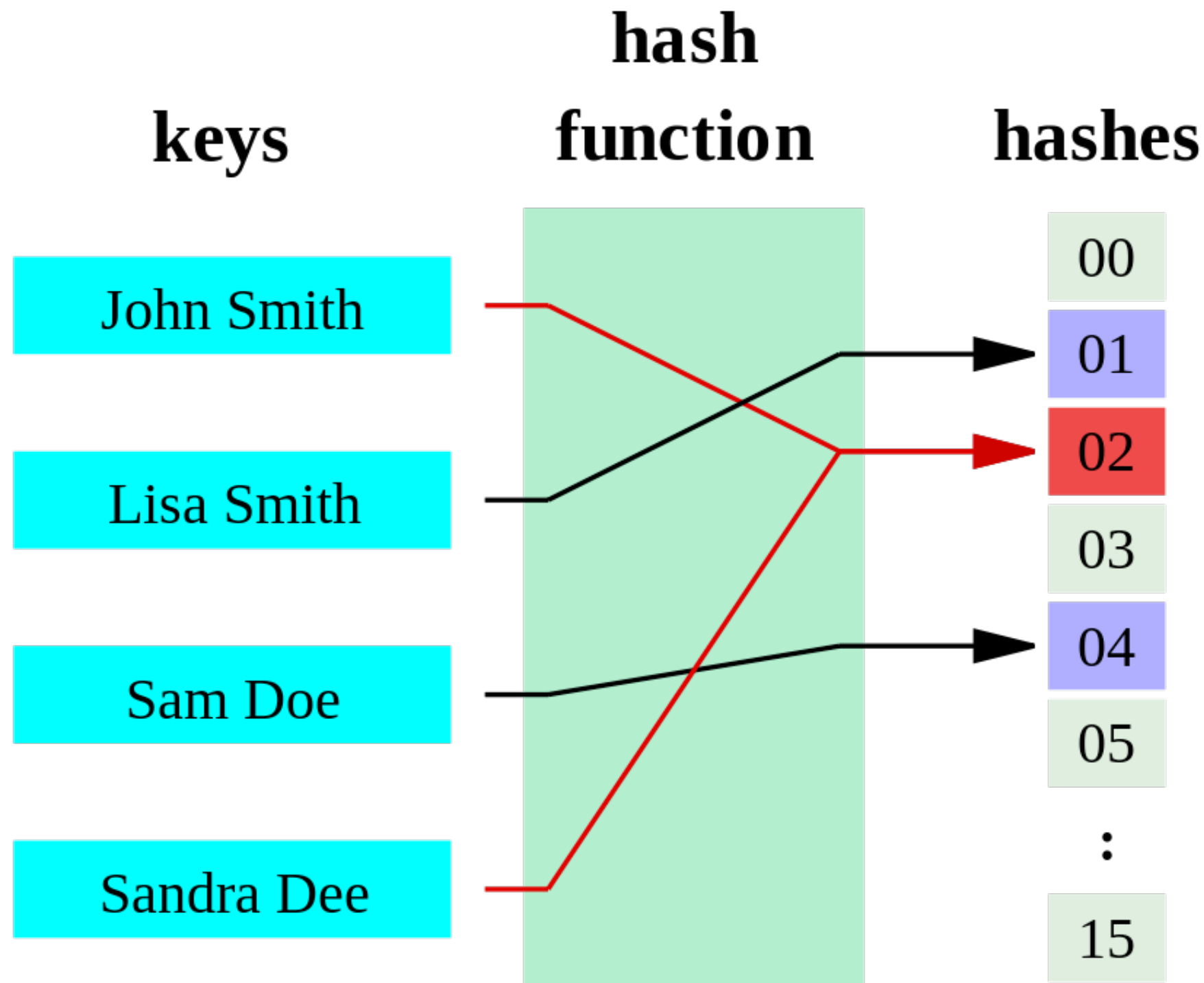


Figure https://en.wikipedia.org/wiki/Hash_function

2018-02-22

A hash function that maps names to integers from 0 to 15. There is a **collision** between keys "John Smith" and "Sandra Dee".

Hashed page tables

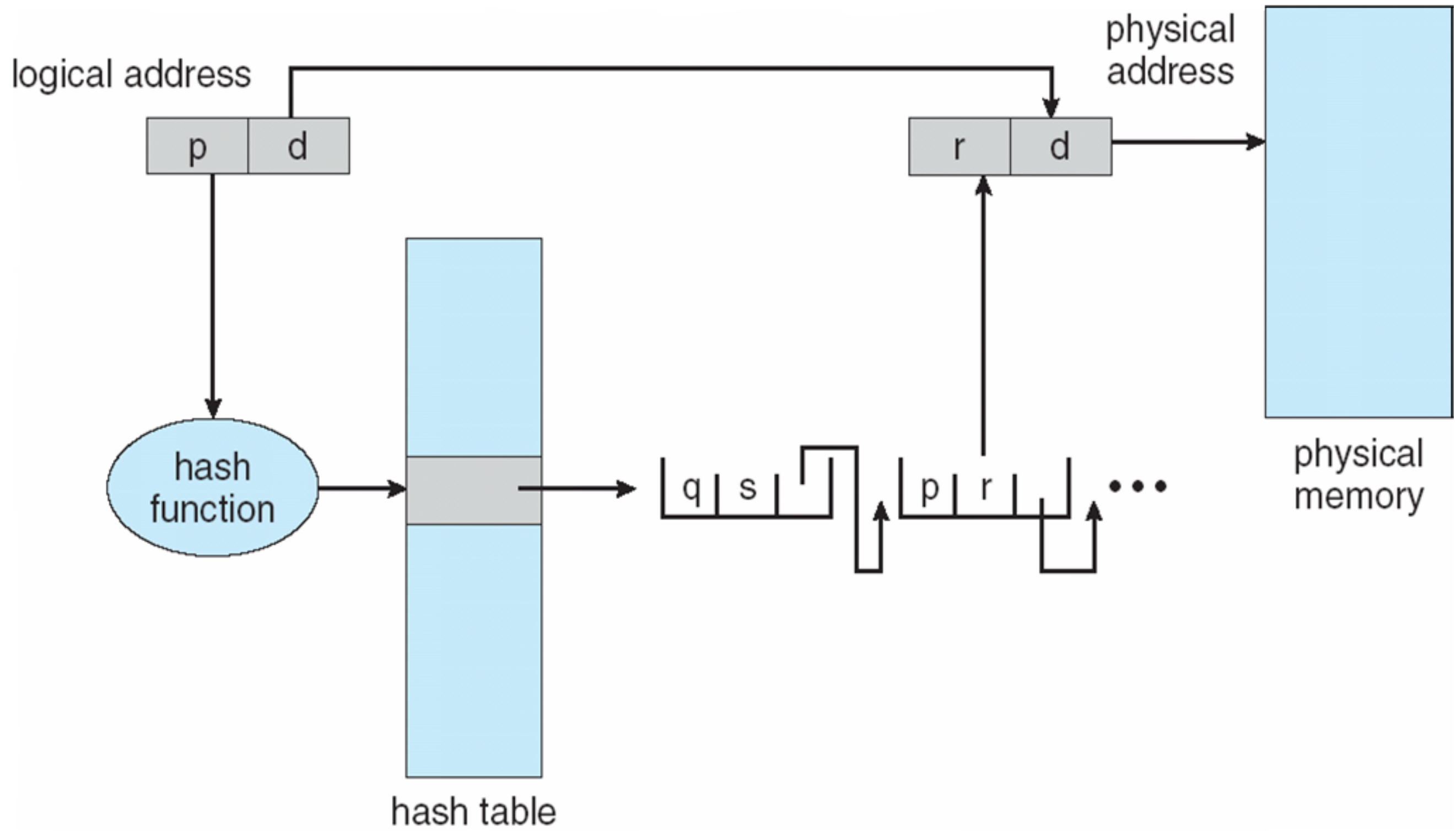
Common in address spaces > 32 bits.

The virtual page number is hashed into a page table.

- ▶ This page table contains a chain of elements hashing to the same location.

Virtual page numbers are compared in this chain searching for a match.

If a match is found, the corresponding physical frame is extracted.



Observations

- ★ Usually, each process has an associated page table.
- ★ The page table has one entry for each page that the process is using.
- ★ The page table is sorted by logical address (seen by the CPU).
- ★ Translation from logical to physical is simple, just find the entry in the page table ...
- ★ Major drawback, page tables can consume lot of memory themselves.

Alternative

- ★ Instead of one page table per process, use one global table.
- ★ One entry for each real frame (ordinary page tables has one entry for each logical page).
- ★ Each entry holds the logical address of a frame.

Inverted page table

Instead of one page table per process, use one global table.

One entry for each real frame.

Each entry holds the logical address of a frame.

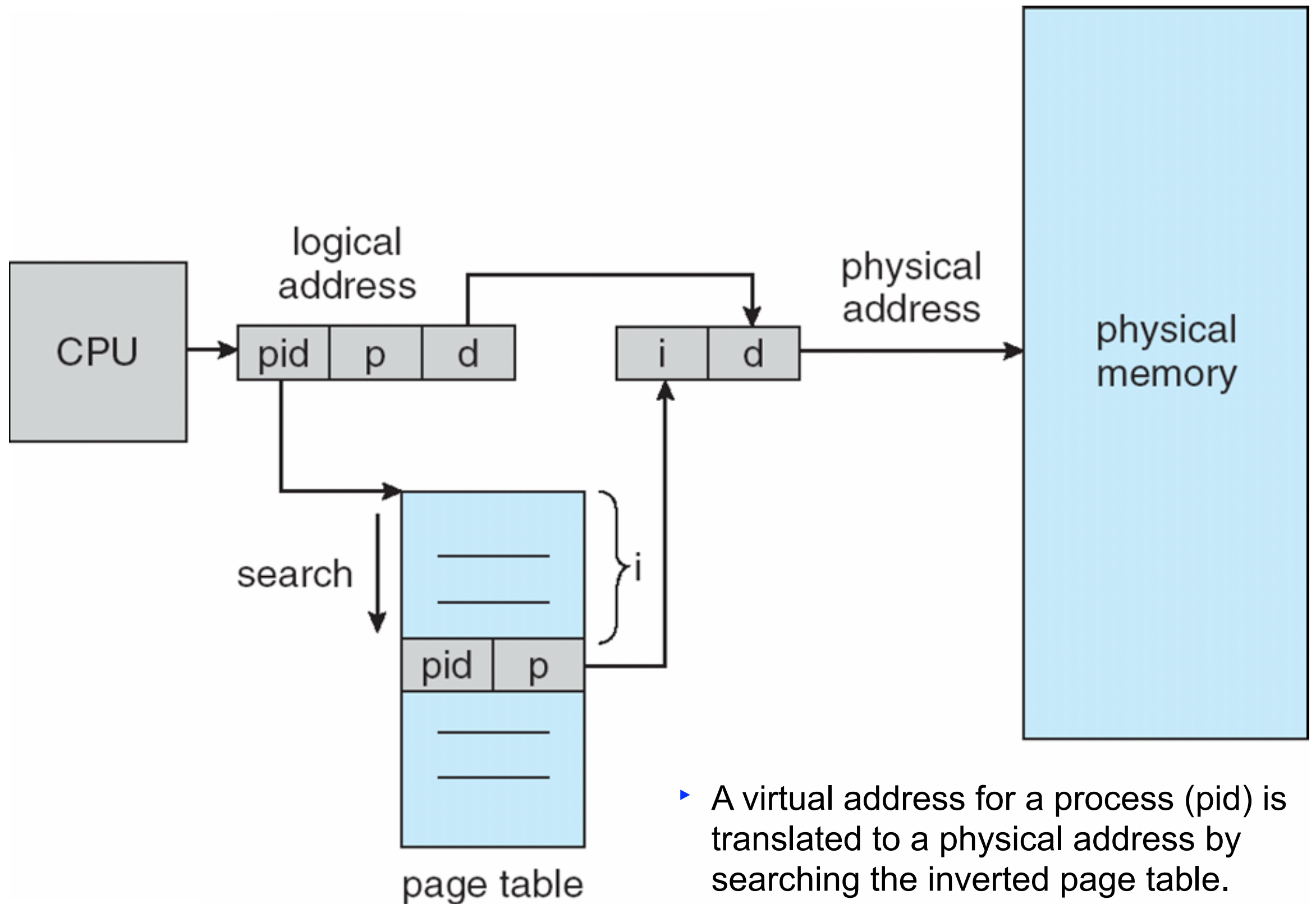
Inverted page table (1)

A single page table for all processes!

One entry for each frame of memory.

Entry consists of a page number (p) and information (pid) about the process that owns that page.

- ▶ **Decreases *memory* needed** to store each page table.
- ▶ **Increases *time* needed** to search the page table when a page reference occurs.



- ▶ A virtual address for a process (pid) is translated to a physical address by searching the inverted page table.
- ▶ When the entry for the process (pid) and page (p) is found the index (i) in the page table is equal to the physical frame number.

Inverted page table (2)

Use **hash table** to limit the search to one or at most a few page-table entries.